

EXPERIMENT NO. 4

Implement a dark/light mode toggle using CSS Custom Properties (Variables)

Name: Rohit Pujari Roll No.: 28 Date: _____

Aim:

To build “RecipeBox”, a recipe listing page whose complete colour scheme is stored in CSS custom properties with two sets of values selected by a data-theme attribute on the <html> element; to provide a three-way Light / Dark / System switch that can follow the operating-system setting through matchMedia(), to remember the user's choice in localStorage and to apply the saved theme before the first paint so that the page never flashes in the wrong colours.

Theory:

CSS custom properties (CSS variables) are properties whose names begin with two hyphens, for example --accent. They are declared like any other property, read with the var() function, inherited by child elements and follow the normal cascade, so a more specific rule can give them a new value. Unlike variables of a pre-processor such as Sass, they exist while the page is running, so when their value changes the browser immediately re-styles every element that uses them.

Two variable sets selected by one attribute

RecipeBox declares all its design tokens once on :root. A second rule, :root[data-theme="dark"], has a higher specificity because of the attribute selector and redefines only the colour tokens. No component rule contains a colour code; the header, cards and tags only use var(--surface), var(--text), var(--accent) and so on. Switching the theme therefore means changing a single attribute: document.documentElement.dataset.theme = 'dark'.

Token	Light theme	Dark theme	Used for
--bg	#fbf7f1	#15120f	Page background
--surface	#ffffff	#211c17	Header bar and recipe cards
--text	#2b2118	#f2ebe3	Recipe names and body text
--muted	#7a6a5c	#b0a193	Cooking time and status line
--accent	#b4450c	#fb923c	Logo, selected button, tag text
--border	#e8dccc	#3a3129	Card and header borders
--shadow	soft shadow	none	Card elevation (shadows are hard to see on dark)

Tokens that do not depend on the theme, such as --radius and --gap, are written only once and are shared by both themes. The colour pairs were chosen for readability: body text on the background has a contrast ratio of 14.8:1 in the light theme and 15.8:1 in the dark theme, far above the WCAG AA minimum of 4.5:1.

Three modes: Light, Dark and System

- **Mode and theme are different:** the mode is what the user selected (light, dark or system) and is stored in localStorage; the theme is what is actually applied (light or dark).
- **System mode:** window.matchMedia('(prefers-color-scheme: dark)') returns a MediaQueryList whose .matches property tells whether the operating system is in dark mode.
- **Live OS changes:** the MediaQueryList fires a change event when the user changes the OS setting while the page is open. RecipeBox re-renders only if the saved mode is still “system”, so an explicit Light or Dark choice is never overridden.
- **Accessibility:** the three buttons are inside role="group" with an aria-label, and each button's aria-pressed attribute tells screen readers which mode is selected.

Avoiding the flash of the wrong theme

If the theme were applied by a script at the end of <body>, the browser could paint the light page first and switch a moment later. RecipeBox therefore places a very small inline script inside <head> that reads localStorage and sets data-theme before the body is parsed. The tag <meta name="color-scheme" content="light dark"> tells the browser that both schemes are supported, so scrollbars and form controls are also drawn in the matching style. All localStorage calls are wrapped in try...catch because storage can be blocked, in which case the page falls back to System.

Reading a variable back

`getComputedStyle(document.documentElement).getPropertyValue('--accent')` returns the value that is currently in effect. The status line at the bottom of the page prints the mode, the applied theme and this value, which proves that the same variable name resolves to a different colour in each theme.

Testing the toggle

Test	Expected result	Observed
First visit, OS in light mode	Mode System, light theme applied	As expected
Click "Dark"	data-theme="dark", Dark button pressed	As expected
Reload after choosing Dark	Saved mode read in <head>, page opens dark	As expected
System selected, OS switched to dark	change event switches page to dark	As expected (Fig. 4.2)
Dark selected, OS switched to light	Page stays dark (explicit choice wins)	As expected

Important Code:

CSS - design tokens and the dark variable set

```
:root {
  --bg: #fbf7f1; --surface: #ffffff; --text: #2b2118; --muted: #7a6a5c;
  --accent: #b4450c; --accent-soft: #fde8d7; --border: #e8dccc;
  --shadow: 0 2px 10px rgb(60 30 0 / .08);
  --radius: 12px; --gap: 16px;
}
:root[data-theme="dark"] {
  --bg: #15120f; --surface: #211c17; --text: #f2ebe3; --muted: #b0a193;
  --accent: #fb923c; --accent-soft: #3b2616; --border: #3a3129;
  --shadow: none;
}
```

HTML <head> - apply the saved theme before the first paint

```
<script>
  // Runs before the page is painted, so a saved dark theme never flashes white.
  (function () {
    var mode = 'system';
    try { mode = localStorage.getItem('recipebox-theme') || 'system'; } catch (e) {}
    var dark = mode === 'dark' || (mode === 'system' && matchMedia('(prefers-color-scheme:
dark)').matches);
    document.documentElement.dataset.theme = dark ? 'dark' : 'light';
  })();
</script>
```

JavaScript - switch, save and follow the OS setting

```
function readMode() {
  try { return localStorage.getItem(KEY) || 'system'; } catch (e) { return 'system'; }
}

function render(mode) {
  const theme = mode === 'system' ? (osDark.matches ? 'dark' : 'light') : mode;
  root.dataset.theme = theme; // CSS switches variable set
  buttons.forEach(b => b.setAttribute('aria-pressed', String(b.dataset.mode === mode)));
  const accent = getComputedStyle(root).getPropertyValue('--accent').trim();
  status.textContent = `mode: ${mode} | applied theme: ${theme} | --accent = ${accent}`;
}

buttons.forEach(b => b.addEventListener('click', () => {
  try { localStorage.setItem(KEY, b.dataset.mode); } catch (e) {}
  render(b.dataset.mode);
}));
// When the user follows the OS setting, react to the OS changing while the page is open.
osDark.addEventListener('change', () => { if (readMode() === 'system') render('system'); });
render(readMode());
```

Output:

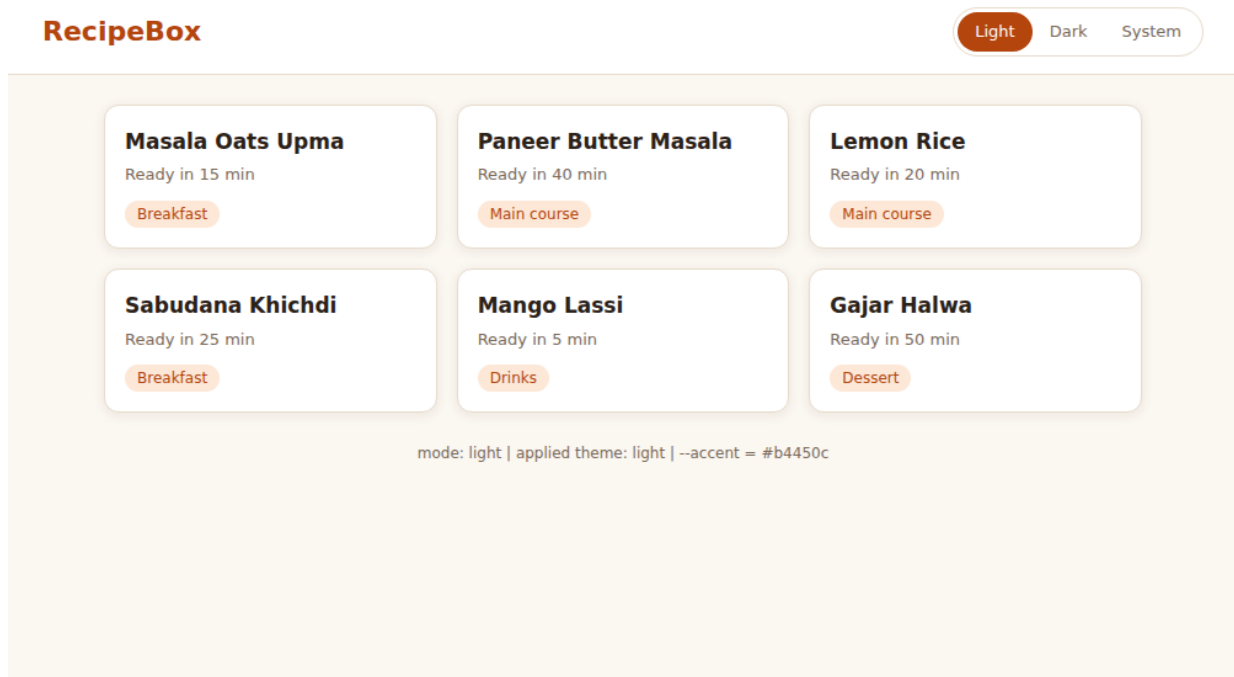


Figure 4.1: RecipeBox with “Light” selected (applied theme light, --accent = #b4450c)

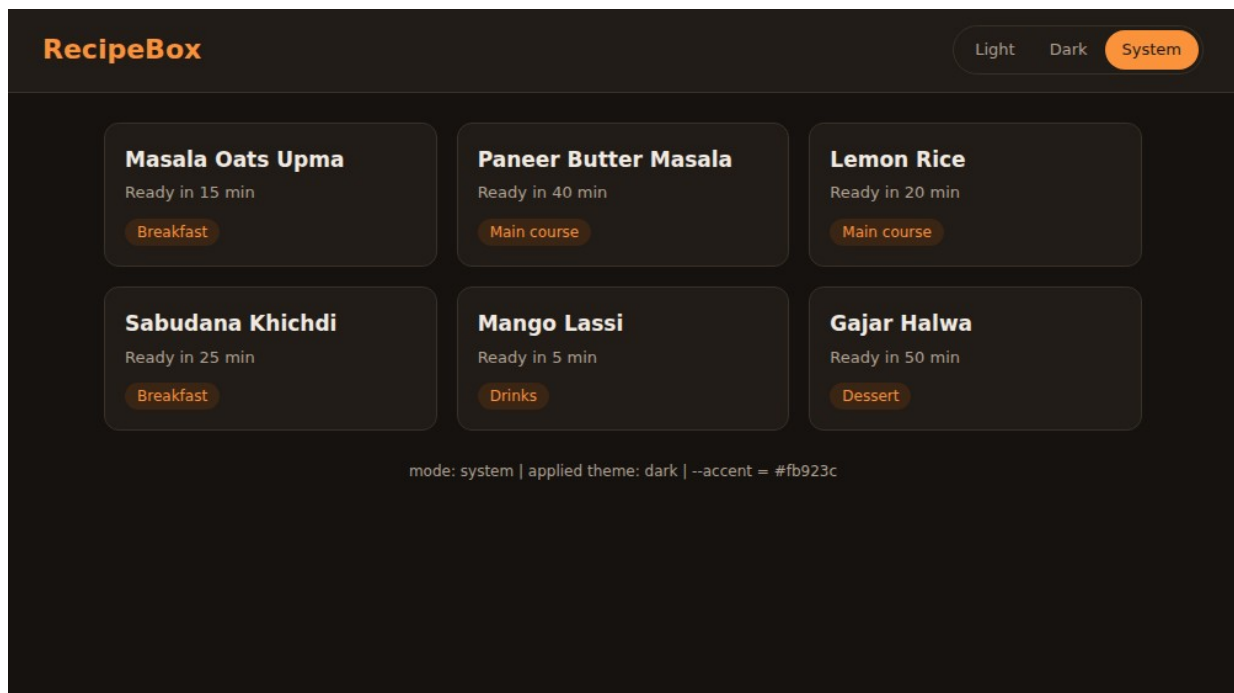


Figure 4.2: “System” selected while the operating system is in dark mode (applied theme dark, --accent = #fb923c)

Conclusion:

A recipe page was themed entirely with CSS custom properties, where a single data-theme attribute on the root element selects the dark set of values. The three-way switch supported Light, Dark and System modes, `matchMedia()` and its change event followed the operating-system preference, `localStorage` remembered the choice and an inline script in `<head>` applied it before the first paint.